

EE-170L Computer Programming Lab

Laboratory Report — No. 09

Introduction to Functions in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	9
Topic	Function Prototype, Definition, Call
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

- Understand the purpose of functions in splitting programs into named, reusable sections.
- Identify the three components of a function: prototype, definition, and function call.
- Write void functions that take no parameters and return nothing.
- Write functions that accept parameters and return computed values.
- Pass arrays and their size as function arguments for array processing.
- Compute and display averages by combining function return values with main() logic.

2. Theoretical Background

2.1 Introduction to Functions

Functions allow a long program to be divided into named, reusable sections. A function has a name identifier, may accept parameters, and may return a result. C++ does not care about function order in the source file as long as the function name is declared via a prototype before it is called.

The most common function encountered so far is main(). Library functions like cout and cin are also functions — we call them without knowing their internal code. When we define our own functions, the code inside executes when the function is called, and control returns to the caller when finished.

If a function is declared with a return type (e.g., int or float), it must return a value of that type. If the return type is void, no return statement is required.

2.2 Prototype of a Function

A function prototype declares the function's name, parameter types, and return type to the compiler before the full definition appears. It ends with a semicolon:

Syntax:

```
returnType functionName(paramType1, paramType2, ...);  
  
// Examples:  
int main();
```

```
float divide(int a, int b);
void square(float x);
```

- void — function returns no value.
- int, float — specifies the data type of the returned value.
- Parameters are listed with their type; names are optional in the prototype.

2.3 Function Definition and Call

The function definition contains the actual executable code. The function call triggers execution. Together with the prototype, these form the three-part structure every C++ function uses:

```
void asterisk();           // prototype

int main()
{
    asterisk();           // call
    return 0;
}

void asterisk()           // definition
{
    for (int i = 0; i < 10; i++)
        cout << "*";
}
```

3. Example Programs

3.1 Function with One Parameter

The asterisk(int n) function takes a single integer and prints that many asterisks in a row:

```
#include <iostream>
using namespace std;
void asterisk(int n);
int main()
{
    asterisk(12);
    return 0;
}
void asterisk(int num)
{
    for (int i = 0; i < num; i++)
        cout << "*";
}
```

Output:

```
*****
```

3.2 Function with Two Parameters and Return Value

The add(int, int) function accepts two integers, computes their sum, and returns it to the caller:

```
#include <iostream>
using namespace std;
int add(int, int);
int main()
{
    int result = add(3, 5);
    cout << result;
    return 0;
}
```

```

}
int add(int num1, int num2)
{
    return num1 + num2;
}

```

Output:

```
8
```

4. Lab Tasks & Solutions

Task 1 — greet() — No Parameters, No Return Value

Declare a void function 'greet' that takes no parameters. Inside it, display a greeting message. Call it from main().

```

#include <iostream>
using namespace std;
void greet();
int main()
{
    greet();
    return 0;
}
void greet()
{
    cout << "Welcome to EE-170L Computer Programming Lab!" << endl;
    cout << "Have a productive lab session." << endl;
}

```

Output:

```

Welcome to EE-170L Computer Programming Lab!
Have a productive lab session.

```

Task 2 — sayHello() Function

Declare a void function 'sayHello' that takes no parameters. Display the message 'Hello, World!'. Call it from main().

```

#include <iostream>
using namespace std;
void sayHello();
int main()
{
    sayHello();
    return 0;
}
void sayHello()
{
    cout << "Hello, World!" << endl;
}

```

Output:

```
Hello, World!
```

Task 3 — calculateSum() — Array + Function + Average

Take 10 integers from the user and store in a 1D array. Create a function calculateSum(int arr[], int size) that returns the total. In main(), call it and compute and display the average.

```

#include <iostream>
using namespace std;
int calculateSum(int arr[], int size);

```

```

int main()
{
    int nums[10];
    cout << "Enter 10 integers:" << endl;
    for (int i = 0; i < 10; i++)
    {
        cout << "Value " << i+1 << ": ";
        cin >> nums[i];
    }
    int sum = calculateSum(nums, 10);
    double avg = (double)sum / 10;
    cout << "Sum      = " << sum << endl;
    cout << "Average = " << avg << endl;
    return 0;
}

int calculateSum(int arr[], int size)
{
    int total = 0;
    for (int i = 0; i < size; i++) total += arr[i];
    return total;
}

```

How it works:

- The array and its size are passed to calculateSum as arguments.
- calculateSum loops through all elements and returns the accumulated total.
- main() receives the sum, divides by 10.0 for the average, and displays both.
- (double) cast ensures decimal precision when computing the average.

5. Function Components — Quick Reference

Component	Location	Ends With	Purpose
Prototype	Before main()	Semicolon	Declares name, params, return type to compiler
Call	Inside main() or another function	Semicolon	Executes the function
Definition	After main() (or before it)	Closing }	Contains the actual body of code

6. Conclusion

This lab introduced functions in C++ — the building block of modular programming. Key takeaways:

- **Functions:** Break programs into named, reusable sections that improve readability and reduce code repetition.
- **Three Parts:** Every function requires a prototype (declaration), a definition (body), and one or more calls.
- **void Functions:** Functions with no return value use void as the return type and need no return statement.
- **Return Values:** Functions typed with int, float, etc. must return a value of that type using the return statement.

- **Array Parameters:** Arrays are passed implicitly by reference — the function works directly on the original array.

Functions are the foundation of structured programming and essential preparation for classes, pointers, and file I/O in upcoming labs.